

A Jai-inspired systems language in Rust. Incremental salsa front end, lossless CST, typed SSA mid-end, two execution engines held byte-identical by a differential harness.

6/6 gates green 919 tests ADR-0065 latest W3 in progress · backtraces the last feature

TESTS	CORPUS	ADRS	DIAGNOSTICS	EDITOR CHECKS
919	149	65	88	156
workspace, all passing	jr files, both engines	0001 to 0065, immutable	codes, E0258 next free	Neovim, verified not gated

The vertical slice, and what it costs to be honest about it

EXIT CRITERIA DONE hello.jr runs in the VM DONE compiles to a native binary DONE rustc-grade diagnostics DONE formatter round-trips the corpus DONE editor integration (Neovim) OPEN verified Linux x86-64 CI run The last one needs a push. Configured, never run, so it is a decision rather than a technical gap.	HOW CORRECTNESS IS ESTABLISHED Both engines run every corpus program and must agree, on output and on trap wording, from one shared formatter. MIR snapshots pin the exact instruction sequence. This wave the basic-module snapshot shows <code>talloc</code> reading the two new context fields and bumping the cursor with pointer arithmetic — the whole allocator, and no new MIR node, because it is three prior waves composed. A feature that adds no machinery is the payoff of the ones that did. Temporary storage is a <code>malloc</code>'d region, a cursor moved by pointer arithmetic, and two context fields — the compiler change is two entries in one array, and everything else is Basic code. That is what the allocator and pointer-arithmetic waves were building toward. A limit stated is better than a limit faked. <code>talloc</code> returns a <code>*u8</code>, so an arena hands out byte buffers only — storing a wider type needs a pointer cast the language does not have yet. The corpus stores bytes and says so, rather than pretending a wider store works.
--	--

The language today

WORKS, END TO END Full integer tower, bool, string, pointers float32 and float64, plain IEEE-754, no traps struct, nominal, one level union, nominal, untagged, fields at offset 0 enum and enum_flags, namespaced, bare dot-member Fixed arrays and views, bounds-checked, returnable cast and xx from context; operator overloading Trapping arithmetic, wrapping variants, bitwise if, else, while, for, break, continue, defer, using Multiple returns, named args, literal defaults import, foreign, system_library, #scope_module One trivial compile-time run, folded Traps name their source line Bounds checks, strippable by build setting or #no_abc context, a hidden parameter passed by pointer Procedures as values: call, pass, return, struct field null, a context-typed pointer literal; malloc and free An allocator in the context, installed and called through Pointer offset <code>p + n</code>, <code>n + p</code>, <code>p - n</code> — element-scaled, unchecked push_context, a block with its own copy of the context talloc / reset_temporary_storage — a per-context bump arena	ABSENT (WAVE) pointer difference <code>p - q</code>, deferred percent on floats, <code>is_nan</code>, math (W7) a tagged variant type (W4.5) explicit backing type; a switch (W4.5) array literals, sub-slicing unary, index and call overloading transmute; float printing #must; a multi-result call in a return polymorphs, macros (W5) arbitrary run, RTTI, insert (W4) backtraces (W3) a per-index #no_abc; any other build setting a cross-file or foreign proc value; comparing one cast to a pointer the difference <code>p - q</code>; <code>p[n]</code>; ordering hands out <code>*u8</code> only; alignment; a growable region
--	--

No error-handling model yet — ADR-0008 reserves the slot and the first half exists (several return values); #must is owed its own ADR. No GC and no RAIi, which is a design value rather than a gap. A union is untagged, so reading a field other than the one last written reinterprets bits: documented in three places because it cannot be diagnosed. No indirect calls, which is the single largest gap in the project and is what blocks the rest of W3.

Compiler internals, 17 crates

STAGE	STATE	NOTE
Lexer, parser, CST, typed AST	works	Hand-written, error-recovering, trivia-preserving
Formatter	works	Pure function over the CST; lost source in 7 of the last 8 waves
HIR, name resolution, modules	works	Flat import merge; cycles legal; export filtering
InternPool: types, values, layout	works	One layout computation and one integer evaluator, shared
Sema: signatures, checking	works	E0212 to E0257; no const-eval here, by design
MIR: typed SSA	works	Block parameters, notphis; explicit bounds check and zeroing
Mid-end	5 passes	Inline, forwarding, const-prop, DCE, plus the bounds-check strip
Const-eval	works	Runs MIR through the bytecode VM
VM: register bytecode, libffi	works	No JIT; indirect calls, and malloc from its own region
Cranelift back end	works	Aggregate returns via sret; indirect calls via func_addr
LLVM back end	not started	W8 owns it
Language server	12 caps	Diagnostics, hover, goto, completion, rename, actions, hints
Neovim integration	works	Runtimepath dir, no plugin manager; 156 checks
Driver	stub	Should consume the workspace notion that now exists

Roadmap: W1 and W2 closed, W3 started

W1 Data	done	Numeric tower, wrapping ops, enum, enum_flags, union, arrays, views, cast, xx, operator overloading. Closed by ADR-0048.
W2 Flow and scope	done	for with it and it_index, labelled break, defer, using, multiple returns, named and default arguments, #scope_module. Closed by ADR-0054; ADR-0055 then closed a gap six corpus files had carried for eleven waves.
W3 Runtime core	in progress	Data structures done; one feature left. context (ADR-0057), the bounds-check build setting (ADR-0058, finishing ADR-0003), indirect calls (ADR-0059), null plus a memory source (ADR-0060/0061), the allocator protocol (ADR-0062), push_context (ADR-0063), pointer arithmetic (ADR-0064), and temporary storage (ADR-0065) — talloc hands out bytes from a per-context bump arena, W3's last data structure and one that composes the previous three waves rather than adding machinery. Remaining: traps with backtraces, the only W3 feature left.
W4 Comptime	not started	Arbitrary compile-time execution, RTTI and Type values, insert, code. PLAN section 5 names this the project's top risk: sema and comptime become mutually recursive.
W4.5 Pattern matching	not started	switch with exhaustiveness, a bare dot-member as a case, and a tagged variant type. Was missing from the wave table entirely — two accepted ADRs deferred decisions to it while no wave scheduled it.
W5 Polymorphism	not started	Polymorphic parameters, modify, bake_arguments, expand macros with hygiene, instantiation caching.
W6 Metaprogram	not started	Workspaces, the compiler message loop, build scripts replacing makefiles, note attributes.
W7 Stdlib	not started	Written in Jairs: String, dynamic array, hash table, Sort, Math, Random, File.
W8 Performance	not started	LLVM via inkwell for release builds, inliner maturity, struct-of-arrays, SIMD, parallel sema. Also owns the compile-throughput number, still unpublished.
W9 Tooling depth	not started	Semantic tokens are the one LSP capability still missing; the other twelve landed early.
W10 Graphics, in Jairs	not started	Window creation through foreign calls, Metal then Vulkan, immediate-mode 2D.

What this session did, and how each wave was scoped

SEVENTEEN WAVES SHIPPED

ADR-0049 through 0065: for and defer, using, aggregate returns, multiple returns, named and default arguments, scope visibility, imported constants, float constants, context, the bounds-check build setting, indirect calls, null plus a memory source, the allocator protocol, push_context, pointer arithmetic, and temporary storage.

Test count 900 to 919. Corpus 116 to 149 files. Neovim checks 103 to 156. W2 closed; W3 opened and all its data structures landed — an allocator in the context, push_context to scope it, raw pointer arithmetic, and a temporary-storage arena built from all three. Only backtraces remain.

Each wave was scoped by writing the feature and seeing what the compiler refused, not by reading the handoff. That found gaps the handoff had missed in both directions: a proc-pointer struct field it called absent (it worked), and a void-returning proc-pointer type it never mentioned (unspellable, and blocking).

TWO CLAIMS THE CODE DISPROVED

An ADR corrected mid-wave. ADR-0060 §4 asserted the VM dereferences a malloc'd pointer via libffi. Running the corpus file the same ADR promised would pass disproved it: the VM's memory is a linear region and a host address is not an offset into it. ADR-0061 corrects it — the VM allocates from its own region, native calls libc, the bits differ and nothing observes them.

A diagnostic that could not be acted on. Installing an imported #foreign procedure into an allocator field reported “expected (s64) -> *u8, found (s64) -> *u8” — the same text twice, because the types differ only in an invisible calling convention. It is E0256 now, the code that says to wrap it.

THE THING WORTH YOUR DECISION

Fourteen waves sit uncommitted across five branches. Two waves ago a careless git checkout reverted the tree-sitter grammar nine waves and cost an hour's reconstruction — the concrete price of not committing. Committing each wave as it goes green would bound the damage from any future slip to a single wave.

Recorded as a recommendation, not a change: the authorisation is yours to give.

Sources: PLAN.md §1.5 and §7, the ADR directory, docs/decisions/DECISIONS.md, and the six gates run today. Every number was measured rather than carried forward — the test count from a full workspace run, the corpus count from a file walk, the diagnostic codes from the const E0nnn definitions across the crates, and the editor-check count from a real verify.lua run. Rebuild with typst compile jairs-dashboard.typ.